

PROGRAMMING FOR AI (509) · MASTER 2 AI

CNNs and Transfer Learning on CIFAR-10

From a baseline CNN trained from scratch to a fine-tuned ResNet-18

Mohammed Yaghi Zaiter

PyTorch · Google Colab T4

PROBLEM FORMULATION

Three questions about CNNs on CIFAR-10

CIFAR-10: 60,000 color images of 32×32 pixels in 10 classes. Which design choices make a CNN better, and when is a pre-trained network the better option?

TASK 1

Baseline from scratch

End-to-end pipeline, learning curves, per-class accuracy and confusion matrix.

TASK 2

What changes the result?

Depth, kernels, stride, BatchNorm, optimizers, learning rate, schedulers and augmentation.

TASK 3

Transfer learning

ResNet-18 from ImageNet: frozen vs fine-tuned, compared with our best CNN.

TASK 1 · PIPELINE

Dataset and pipeline

45,000

train images

5,000

validation: choose the best epoch

10,000

test: only used at the end

Pre-processing

- ToTensor: pixels from $[0, 255]$ to $[0, 1]$
- Normalize with the CIFAR-10 mean / std per channel
- ResNet-18: resize to 128×128 , ImageNet mean / std

Augmentation (train only)

- basic: random crop (padding 4) + flip
- full: + rotation / translation + color jitter
- MixUp: blend two images and their labels

Baseline CNN trained from scratch

Layer	Output shape
Input image	$3 \times 32 \times 32$
conv1: 32 filters + ReLU + pool	$32 \times 16 \times 16$
conv2: 64 filters + ReLU + pool	$64 \times 8 \times 8$
conv3: 128 filters + ReLU + pool	$128 \times 4 \times 4$
Flatten	2048
fc1 + ReLU	256
fc2 (one score per class)	10

620,362

parameters, 85% of them in fc1

 $h_{out} = (h_{in} + 2p - k) / s + 1$

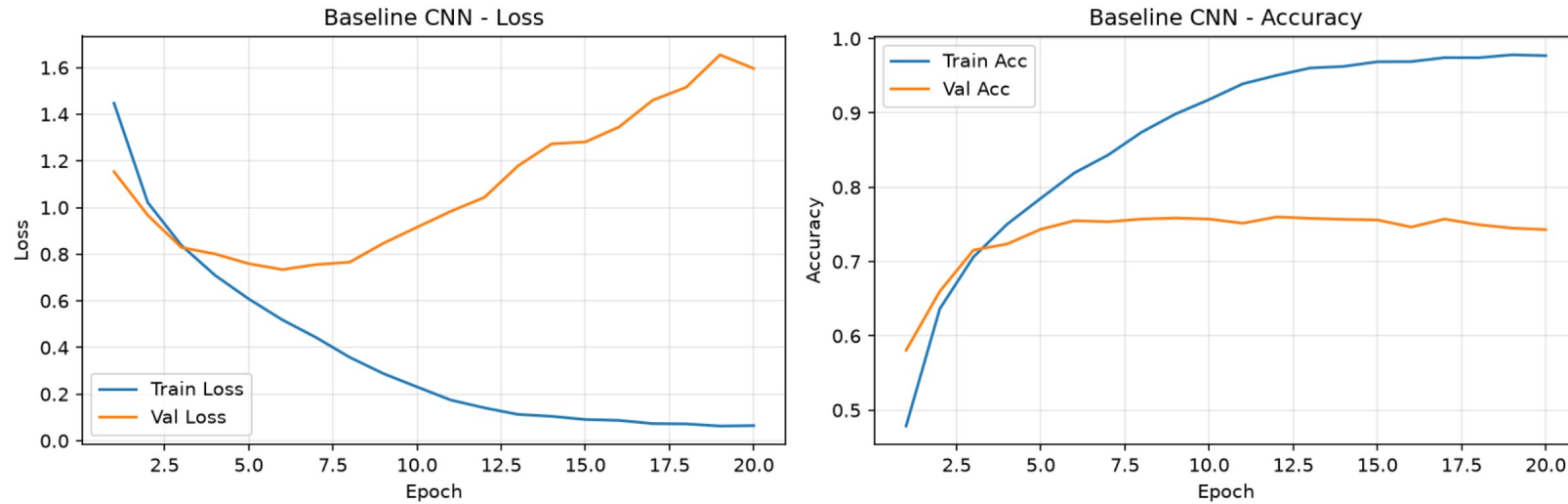
3×3 kernels, padding 1 keep the size; each pool divides it by 2

Adam, lr 0.001, 20 epochs

No BatchNorm, dropout or augmentation on purpose

TASK 1 · LEARNING CURVES

The baseline overfits after epoch 6



74.9%

test accuracy (best epoch 12)

95.1% / 76.0%

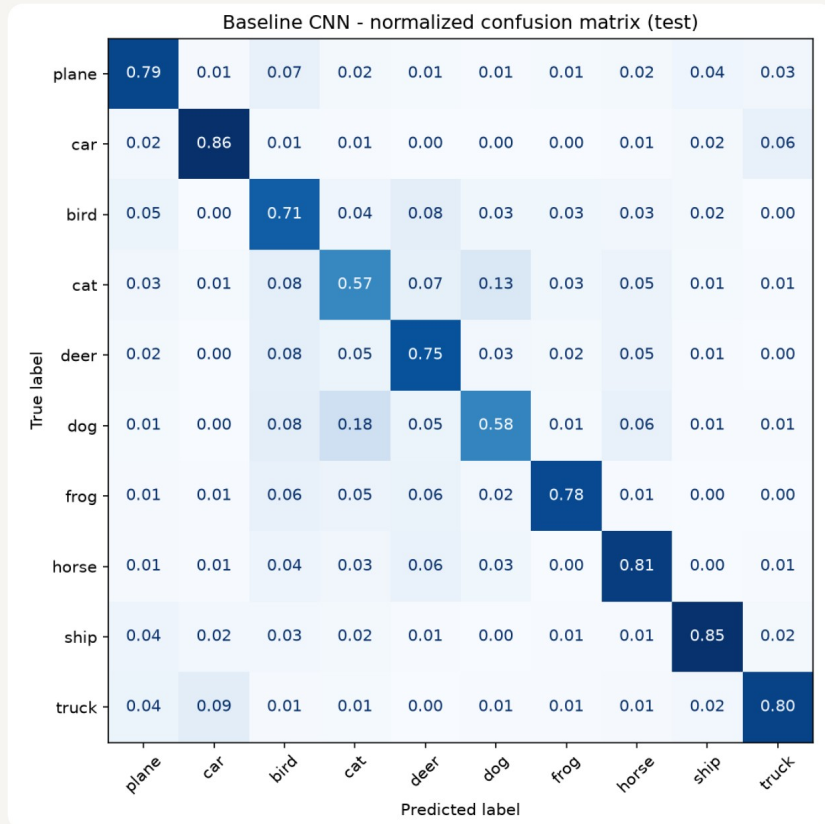
train / validation accuracy

0.19

generalization gap

TASK 1 · CONFUSION MATRIX

Cats and dogs are confused the most



Most common mistakes (test set)

True class → predicted as	Share
dog → cat	18%
cat → dog	13%
truck → car	9%
bird ↔ deer	8%

Hardest: cat 57%, dog 58%. Easiest: car 86%, ship 85%. The confused classes share shapes, colors or backgrounds.

TASK 2 · METHOD

Change one thing at a time

Each run = baseline + one change · same seed, same split, 20 epochs

Depth

2 / 3 / 5 conv layers, 1 / 2 / 4 linear layers

Kernel size

3×3 / 5×5 / 7×7

Stride and padding

max pool vs stride-2 conv, same vs no padding

Batch normalization

off / on after each conv

Optimizer

SGD + momentum / Adam / AdamW

Learning rate

0.01 / 0.001 / 0.0001 with Adam

Scheduler

none / StepLR / cosine

Augmentation

none / basic / full / full + MixUp

Measured for each run

params, latency, peak val acc, generalization gap

TASK 2 · ARCHITECTURE

BatchNorm helps, big kernels hurt

Change from the baseline	Params	Receptive field	Test acc	vs baseline
Baseline (3 conv, 3×3, max pool)	620K	22 px	75.3%	–
2 conv layers	1.07M	10 px	72.3%	–3.0
5 conv layers	1.05M	94 px	75.5%	+0.2
1 linear layer	114K	22 px	74.1%	–1.2
4 linear layers	1.31M	22 px	74.6%	–0.7
5×5 kernels	786K	36 px	73.3%	–2.0
7×7 kernels	1.03M	50 px	69.4%	–5.9
Stride-2 conv instead of max pool	620K	15 px	67.9%	–7.4 (worst)
No padding	227K	22 px	73.1%	–2.2
BatchNorm after each conv	621K	22 px	77.4%	+2.1 (best)

TASK 2 · OPTIMIZATION

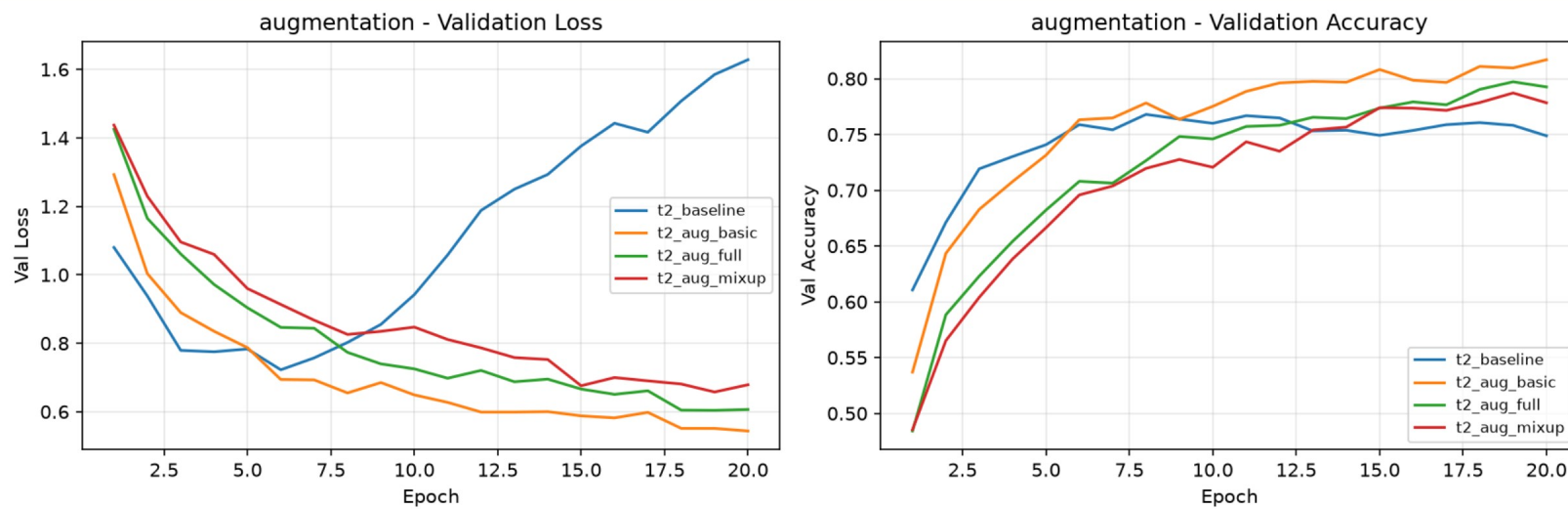
The learning rate matters most

Setting	Test acc	vs baseline	Note
Adam, lr 0.001 (baseline)	75.3%	–	
SGD + momentum, lr 0.01	75.9%	+0.6	98.7% train acc
AdamW, lr 0.001	75.5%	+0.2	smaller gap
Adam, lr 0.01	10.0%	–65.3	did not train
Adam, lr 0.0001	69.4%	–5.9	too slow
StepLR ($\div 10$ every 6 epochs)	77.3%	+2.0	
Cosine annealing	76.9%	+1.6	

lr = 0.01: the loss stays at $2.303 = \ln(10)$, the loss of a random guess. The model predicts one class for every image.

TASK 2 • AUGMENTATION

Augmentation gives the biggest gain



80.5%

crop + flip, +5.2 points

0.13 → 0.007

generalization gap

2× slower

full aug: 32 s vs 14 s per epoch

BEST CNN FROM SCRATCH

82.4%

test accuracy, +7.1 points over the baseline

The combined recipe

- 64 → 128 → 256 filters + BatchNorm
- Dropout 0.3
- AdamW + cosine scheduler
- Full augmentation, 30 epochs
- Val loss 0.47 (baseline 1.63)

TASK 3 • SETUPS

Reusing a ResNet-18 trained on ImageNet

Method	What is trained	Trainable	lr
CNN from scratch	all layers, random start	1.42M	0.001
Frozen	new head only (requires_grad = False)	5,130	0.001
Fine-tune layer4	last residual block + head	8.40M	0.0001
Fine-tune all	the whole network	11.18M	0.0001

Fine-tuning uses a 10× smaller learning rate so the pre-trained weights are adjusted, not destroyed.

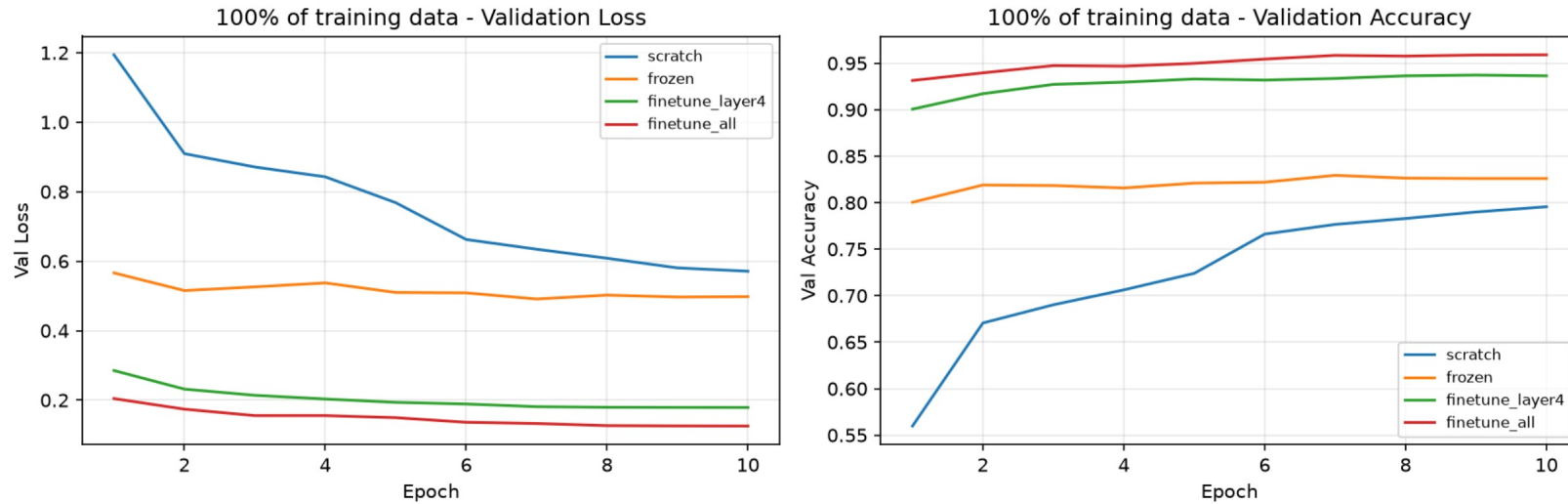
128×128 input, ImageNet normalization

AdamW + cosine, basic augmentation, 10 epochs

Each trained on 10% and 100% of the data

TASK 3 • RESULTS AND CONVERGENCE

Fine-tuning reaches 95.6% test accuracy



79.1%

CNN from scratch

82.7%

frozen ResNet-18

93.1%

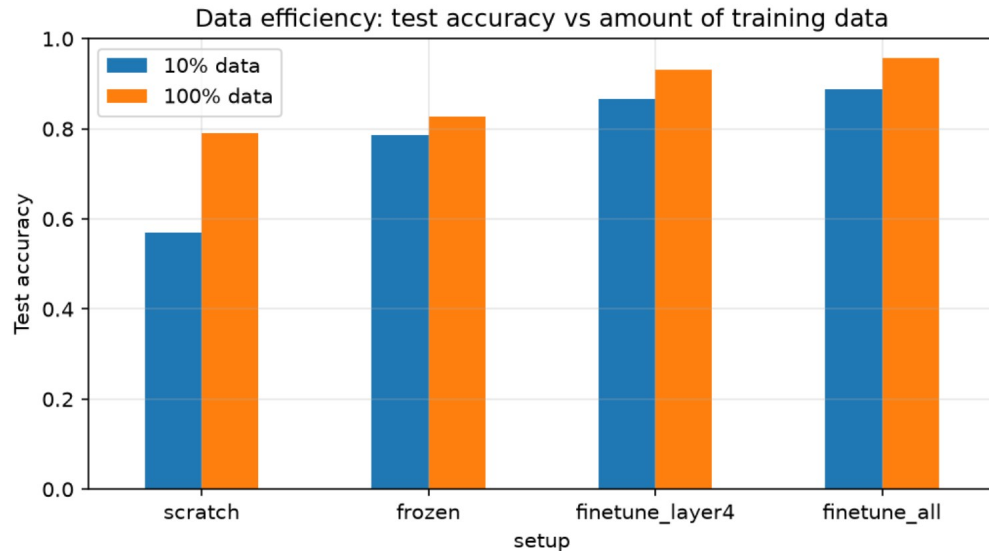
fine-tune layer4

95.6%

fine-tune all

TASK 3 · DATA EFFICIENCY

Transfer learning needs much less data



Test acc	10%	100%	Drop
From scratch	57.0%	79.1%	-22.2
Frozen	78.6%	82.7%	-4.0
Layer4	86.7%	93.1%	-6.4
Fine-tune all	88.9%	95.6%	-6.8

With 4,500 images, full fine-tuning (88.9%) beats our best CNN trained on all 45,000 images (82.4%).

CONCLUSION

Key findings

- 1 The baseline overfits: 95% train vs 76% validation accuracy.
- 2 Reducing overfitting helps more than adding capacity: augmentation +5.2, BatchNorm +2.1, scheduler +2.0 points. Big kernels and strided convs hurt.
- 3 The learning rate is critical: 0.01 did not train at all, 0.0001 was too slow.
- 4 Combined, the CNN from scratch reaches 82.4% (+7.1 points).
- 5 Fine-tuned ResNet-18 reaches 95.6%, converges in 1 epoch and keeps 88.9% with 10% of the data, but it is 8× bigger and 3× slower per epoch.

Load a checkpoint and test it

1 • Choose

Set CHECKPOINT_NAME to any of the 29 saved .pth files

2 • Load

Rebuild the model, load its state_dict, switch to eval mode

3 • Test

Random test mini-batch, then the full 10,000-image test set

Thank you. Questions?